8주차: 그리디와 혼합 평가

한 번의 스캔으로 가능한 결정

위치마다 이동 가능한 거리가 다를 때 목적지 도달 여부와 최소 이동 횟수를 구합니다. 순환 경로에서는 연료가 바닥나지 않는 출발점을 찾습니다.

모든 선택을 저장하지 않고도 답을 구하려면 버린 후보가 답이 될 수 없다는 근거가 필요합니다. 도달 범위와 누적합으로 그 근거를 확인합니다.

8주차 학습 목표

15·16차시: 도달 범위·후보 제거·반례·혼합 평가

- 입력 계약과 경계 조건 확인
- 상태 추적과 Python 구현
- 정당성·시간·보조 공간·출력 공간 설명

도입 상황, 표와 점검 문제는 강의용 추가 예시입니다.

15차시 · 도달 범위와 후보 제거

최대 점프 길이로 도달 여부와 최소 횟수를 구하고 원형 경로의 시작점을 찾습니다. 국소 선택이 버리는 후보를 증명해야 탐욕법을 사용할 수 있습니다.

선수 지식과 Python 문법

range(n-1)은 마지막 인덱스를 처리하지 않습니다. enumerate는 위치·값을 함께 주고 zip(gas,cost)는 대응 값을 묶습니다. zip은 길이가 다르면 짧은 쪽까지만 진행하므로 입력 계약을 확인합니다.

최소 점프의 입력 조건

- 각 값: 허용되는 최대 전진 거리
- 0부터 그 값 이하의 점프 허용
- 공식 45번: 마지막 위치 도달 가능 보장
- 마지막 인덱스에서는 점프 수를 더하지 않음

도달 불가능한 입력까지 받으려면 먼저 `can_jump`로 검사하거나 별도의 실패 반환을 정의해야 합니다.

55 · 문제와 상태

LeetCode 55: Jump Game · Medium

각 위치의 최대 전진 거리로 마지막 인덱스에 도달할 수 있는지 판정합니다.

`nums=[2,0,1,0] → True`

도달 가능한 index에서만 farthest=max(farthest,index+step)을 갱신합니다.

55 · 구현

```
def can_jump(nums):
    farthest = 0
    for index, step in enumerate(nums):
        if index > farthest:
            return False
        farthest = max(farthest, index + step)
    return True
```

55 · 정당성과 비용

이전 도달 집합은 0..farthest의 연속 구간입니다. 도달한 위치에서 최대 step 이내의 모든 거리가 허용되어 같은 형태를 유지합니다. index>farthest면 건널 방법이 없습니다.

n=길이. O(n) 시간·O(1) 보조 공간.

45 · 문제와 상태

LeetCode 45: Jump Game II · Medium

마지막 위치까지 필요한 최소 점프 수를 반환합니다.

`nums=[2,3,0,1,0] → 2`

end는 현재 점프 수의 끝, farthest는 다음 범위입니다. index==end일 때만 jumps를 늘립니다.

45 · 구현

```
def min_jumps(nums):
    end = farthest = jumps = 0
    for index in range(len(nums) - 1):
        farthest = max(farthest, index + nums[index])
        if index == end:
            jumps += 1
            end = farthest
    return jumps
```

45 · 정당성과 비용

현재 레벨의 모든 도달 위치를 스캔한 뒤 다음 최대 범위를 확정합니다. 이는 암묵적 무가중 그래프의 BFS 레벨 확장이므로 더 적은 점프로 끝을 넘을 수 없습니다.

n=길이. O(n) 시간·O(1) 보조 공간.

134 · 문제와 상태

LeetCode 134: Gas Station · Medium

빈 탱크로 출발해 원형 경로를 한 바퀴 도는 시작점을 반환하고 없으면 -1입니다.

`gas=[1,4,2], cost=[3,1,3] → 1`

차이의 total과 후보 구간 tank를 합산합니다. tank<0이면 i+1부터 다시 시작합니다.

134 · 구현

```
def gas_start(gas, cost):
    total = tank = start = 0
    for index, (supply, use) in enumerate(zip(gas, cost)):
        total += supply - use
        tank += supply - use
        if tank < 0:
            start, tank = index + 1, 0
    return start if total >= 0 else -1
```

134 · 정당성과 비용

s..i에서 처음 실패했다면 s..j-1의 누적합은 음수가 아닙니다. 따라서 중간 j에서 시작한 j..i 합은 실패 합보다 크지 않아 모두 실패합니다. 전체 합이 음수가 아니면 마지막 후보는 끝 구간을 통과하고, 앞의 제거된 구간을 감싸 돌아도 부족하지 않습니다.

n=정류장 수. O(n) 시간·O(1) 보조 공간.

상태 추적

i (nums=2,3,0,1,0)	farthest	end	jumps
0	2	2	1
1	4	2	1
2	4	4	2
3	4	4	2

인덱스 갱신 후 상태이며 현재 범위 끝에서만 점프 수를 늘립니다.

실패 입력과 이해 확인

[2,3,0,0,0]에서 처음부터 최대 거리로 점프하면 위치 2의 0에서 멈춥니다. 도달 가능한 위치 1도 탐색하면 마지막 위치까지 갈 수 있습니다. 알고리즘은 가능한 위치들의 도달 범위를 유지합니다.

성공 예 하나가 탐욕법의 증명인가요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

16차시 · 패턴 비공개 혼합 평가

문제의 주제가 적혀 있지 않은 상태에서 입력·출력·제약부터 정리합니다. 익숙한 문제의 유창성과 새로운 조건으로의 전이를 따로 관찰합니다.

선수 지식과 Python 문법

값 비교·객체 비교, 입력 변경 여부, 반환 타입을 먼저 확인합니다. 함수가 올바른 값만 반환해도 원본 변경 금지 조건을 어기면 계약 위반입니다.

200 · 문제와 상태

LeetCode 200: Number of Islands · Medium

문자 격자의 상하좌우로 연결된 육지 덩어리 수를 반환합니다.

`grid=[["1","0","1"],["1","0","0"]] → 2`

미방문 육지에서 count를 늘리고 연결된 육지를 스택으로 모두 표시합니다.

200 · 구현

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

200 · 정당성과 비용

한 번 시작한 탐색은 연결 경로를 따라 그 성분의 모든 노드와 그 노드만 방문합니다. 이미 표시한 성분은 다시 시작하지 않으므로 시작 횟수가 성분 수입니다.

R,C=행·열 수. O(RC) 시간·O(RC) 보조 공간. 일반 인접 리스트에서는 O(V+E) 시간입니다.

상태 추적

평가 단계	학생 기록	교사 관찰
정의	입력·출력·제약	가정 추가 여부
설계	상태·후보 제거	근거의 일관성
실행	작은 입력·경계	최초 실패 상태
설명	시간·공간	복사·출력·스택 비용

기록에서 처음 잘못된 가정이나 상태를 찾습니다.

실패 입력과 이해 확인

값 비교는 ==입니다. is는 객체 동일성입니다. [[“1”,”0”],[“0”,”1”]]은 대각선을 연결하지 않아 두 성분입니다.

같은 문제 재풀이 성공으로 신규 문제 전이도 입증되나요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

실패 구간의 출발점 제거

i	gas[i]−cost[i]	누적 tank	다음 start
0	-2	-2 → 0	1
1	3	3	1
2	-1	2	1

전체 합은 0. 인덱스 1에서 출발하면 순환 경로를 마칩니다. gas와 cost는 같은 길이이며 탱크 용량 제한은 없습니다.

중간 점검

1. `[2,0,0,1]`에서 마지막 위치에 도달 가능한가요?
2. `[2,3,0,1,0]`의 최소 점프 수는?
3. gas=`[1,4,2]`, cost=`[3,1,3]`의 출발점은?
4. 입력 길이가 1인 도달 가능한 배열의 최소 점프 수는?

독립 실습과 다음 연결

학생 실습

힌트·해설

교사용 진행안

8주차 코드 내려받기

코드·상태 정의·정당성·복잡도·경계 입력을 함께 제출합니다. 8주차에는 주제를 숨긴 혼합 평가가 있습니다.

혼합 평가 진행

혼합 평가 문제에서 익숙한 문제 하나와 새로운 조건의 작업 선택 문제를 풉니다.

- 먼저 입력·출력·제약 기록
- 풀이 선택 근거와 경계 입력 제출
- 제출 후 교사용 해설과 비교

작업 선택 문제는 강의용 추가 문제입니다.

실행과 참고자료

주차별 코드의 code 폴더에서 실행합니다.

```
python demo_15.py
python demo_16.py
python verify.py
python verify.py --student
```

마지막 명령은 학생 파일을 구현한 뒤 실행합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).